

Chapitre 6 Opérateurs

C++ offre une grande richesse d'opérateurs.

Opérateurs arithmétiques

Les opérateurs arithmétiques sont utilisés pour réaliser des calculs simples sur des valeurs.

Opérateur	Dénomination	Fonction	Exemple
+	Addition	Ajoute deux valeurs	3 + 9 retourne 12
−	Soustraction	Soustrait deux valeurs	6.5 − 4 retourne 2.5
*	Multiplication	Multiplie deux valeurs	2 * 5 retourne 10
/	Division	Divise deux valeurs	3.0 / 4.0 retourne 0.75
%	Modulo	Reste de la division entière	8 / 4 retourne 2

Tableau 8 : opérateurs arithmétiques simples.



La division est entière lorsque les deux opérandes sont des entiers. Exemple : $7 / 3 = 2$, $2 / 3 = 0$ (voir paragraphe « Opérations sur les entiers » page 19).

Remarque

- Il est possible d'utiliser comme opérande une variable à la place d'une valeur numérique explicite. Dans ce cas, l'opération est réalisée sur la valeur contenue dans la variable.



- On appelle opérande (n. m.) le ou les termes qui composent une opération. Exemple : dans l'opération $2 + 3$, 2 et 3 sont des opérandes.

Opérateur d'affectation =

L'opérateur d'affectation = est utilisé pour affecter une valeur à une variable.

Syntaxe

```
variable = valeur;
```

ou

```
variable = variable2;
```

Remarques

- Le membre de gauche de l'opérateur = (1^{er} opérande de l'opérateur) est une variable. Le membre de droite de l'opérateur = (2^{ème} opérande de l'opérateur) est une valeur. Lorsqu'une variable est spécifiée comme membre de droite, c'est la valeur de cette variable (son contenu) qui est utilisée par l'opérateur pour réaliser l'affectation.
- Les langages C et C++ sont dits « fortement typés ». Cela signifie que le compilateur vérifie que le type de la valeur correspond rigoureusement au type de la variable pour réaliser l'affectation. Cette non correspondance entraîne une erreur de compilation. On doit avoir :

```
type de la variable = type de la valeur;
```

Changement de type () (cast)

L'opérateur (type) modifie le type d'une valeur vers le type type. On parle de "cast" ou "casting" de valeurs.

Exemple

```
int nN = 0;
double dD = 2.718;

nN = (int) dD;           // valide, cast du contenu de dD en int
cout << nN;             // affiche 2 (partie entière de dD)
```

Remarques

- Lors de la conversion d'un réel en entier, la partie décimale du réel est supprimée. Aucun arrondi n'est réalisé.
- Pour réaliser l'arrondi d'une variable entière, on utilise l'écriture :

```
(int) (dE + 0.5);
```

qui retourne la partie entière de dE tant que sa partie décimale est strictement inférieure à 0.5, et la partie entière de dE+1 dès que sa partie décimale est supérieure ou égale à 0.5.

Exemple

```
(int) (3.49+0.5) retourne 3 et (int) (3,52+0.5) retourne 4
```

cast automatique

L'instruction :

```
double dE = 3;
```

bien qu'ambigüe puisque dE est une variable réelle de type double et que 3 est une variable entière, est néanmoins acceptée, car le compilateur transforme automatiquement la variable entière 3 en variable réelle 3.0. Au final, la ligne exécutée est celle-ci :

```
double dE = (double) 3;
```

Pour lever toute incertitude, il est préférable d'écrire :

```
double dE = 3.0;
```

Opérateurs de manipulation de bits

Les opérateurs de manipulation de bits opèrent sur els bits des variables. Ils sont quasi exclusivement réservés aux variables entières.

Opérateur	Dénomination	Fonction	Exemple
&	ET (AND) bit à bit	ET sur chaque bit de même poids	0110 & 1100 donne 0100
	OU (OR) bit à bit	OU sur chaque bit de même poids	0110 1100 donne 1110
^	OU exclusif bit à bit	OU exclusif	0110 ^ 1100 donne 1010
<< n	Décalage à gauche	décale n fois les bits à gauche	0111 << 1 donne 1110
>> n	Décalage à droite	décale n fois les bits vers la droite	0111 >> 1 donne 1011
>>> n	Décalage à droite	Idem >>, des 0 insérés à gauche	0111 >> 1 donne 0011
~	Complément à un	Complémentation à un unaire	~ 0100 donne 1011

Tableau 9 : opérateurs de manipulation de bits.

Les opérateurs de manipulation de bits sont souvent utilisés avec des variables qui correspondent à des masques utilisés pour identifier chaque bit de la variable à manipuler.

Optimisations

Lorsqu'on réalise une opération sur le contenu d'une variable et que l'on range le résultat dans cette variable, on utilise une version simplifiée et surtout optimisée des opérateurs.

Par exemple, l'écriture :

```
double dX = 3.5;
dX = dX + 2.1;
```

s'écrit :

```
double dX = 3.5;
dX += 2.1;
```

Cette écriture est plus compacte, donc plus rapide à écrire et plus facile à relire, et le code généré est plus rapide.

Opérateur	Usage
+=	additionne deux valeurs et affecte le résultat dans la variable (à gauche)
-=	soustrait deux valeurs et affecte le résultat dans la variable
*=	multiplie deux valeurs et affecte le résultat dans la variable
/=	divise deux valeurs et affecte le résultat dans la variable
%=	Réalsie le modulo de deux valeurs et affecte le résultat dans la variable

Tableau 10 : combinaison des opérateurs arithmétiques et d'affectation.

Le même type de simplification s'applique sur les opérateurs de manipulation de bits :

&=, |=, ^=, ~=, <<=, >>= et >>>=.

Opérateurs d'incrément et de décrémentation

Les opérateurs ++ et -- sont utilisés pour ajouter ou soustraire 1 à une variable entière.

++ : incrément. `i = i + 1;` peut s'écrire `i += 1;`, mais aussi `++i;` (préfixe, pré-incrément) ou `i++;` (suffixe, post-incrément). En préfixe, la variable est incrémentée avant son utilisation, alors qu'en suffixe elle est incrémentée après son utilisation.

Exemple : `i = 3; j = i++;` conduira à initialiser `i` et `j` par 3, puis à ajouter 1 à `i` qui prend la valeur 3, alors que `i = 3; j = ++i;` conduira à initialiser `i` par 3, l'incrémenter de 1 (`i` vaut 4) puis l'affecter à `j` (`j` vaut 4).

-- : décrémentation. `i = i - 1;` peut s'écrire `i -= 1;`, mais aussi `--i;` (préfixe) ou `i--;` (suffixe). En préfixe, la variable est décrémentée avant son utilisation, alors qu'en suffixe elle est décrémentée après son utilisation.

Exemple : `i = 3; j = i--;` conduira à initialiser `i` et `j` par 3, puis à soustraire 1 à `i` qui prend la valeur 2, alors que `i = 3; j = --i;` conduira à initialiser `i` par 3, le décrémenter de 1 (`i` vaut 2) puis l'affecter à `j` (`j` vaut 2).

Opérateurs relationnels

Les opérateurs relationnels mettent en relation deux expressions, et le résultat est une expression booléenne fausse (0) ou vraie (tout entier différent de 0). Voir aussi paragraphe « Opérateurs de comparaison » page 26.

Opérateur	Dénomination	Fonction	Exemple
<code>==</code>	égalité	Vérifie l'égalité de deux valeurs	<code>(dX==3.)</code> retourne 1 si dX vaut 3., 0 sinon
<code>!=</code>	différent	Vérifie si deux valeurs sont différentes	<code>(dX !=3.)</code> retourne 0 si dX vaut 3., 1 sinon
<code><</code>	Infériorité stricte	Vérifie si une valeur est strictement inférieure à une autre	<code>(dX<3.)</code> retourne 1 si dX est strict. inférieur à 3., 0 sinon
<code>></code>	Supériorité stricte	Vérifie si une valeur est strictement supérieure à une autre	<code>(dX>3.)</code> retourne 1 si dX est strict. supérieur à 3, 0 sinon
<code><=</code>	inférieur ou égal	Vérifie si une valeur est inférieure ou égale à une autre	<code>(dX<3.)</code> retourne 1 si dX est inférieur ou égal à 3., 0 sinon
<code>>=</code>	supérieur ou égal	Vérifie si une valeur est supérieure ou égale à une autre	<code>(dX>3.)</code> retourne 1 si dX est supérieur ou égal à 3, 0 sinon

Tableau 11 : opérateurs relationnels.



Attention, en C++, de nombreuses erreurs proviennent de la confusion des opérateurs d'affectation (`=`) et d'égalité (`==`).

Opérateurs logiques

Les opérateurs logiques combinent des expressions booléennes entre elles pour vérifier si plusieurs conditions sont vraies ou fausses. Voir aussi paragraphe « Combinaisons de booléens » page 26.

Les expressions reliées par ces opérateurs sont évaluées de la gauche vers la droite. L'évaluation prend fin dès que le résultat d'une seule expression entraîne un résultat définitif pour l'expression globale.

- !** : négation unaire (NOT). Cet opérateur a pour effet d'inverser la valeur du résultat de l'expression qui le suit.
- &&** : ET logique (AND)
- ||** : OU logique (OR)

Opérateur conditionnel ternaire

L'opérateur conditionnel est un opérateur ternaire mettant en relation trois expressions. L'expression résultante est une valeur. Cet opérateur est composé des deux signes `?` et `:`.

Syntaxe

```
(expression 1) ? expression 2 : expression 3;
```

L'opérateur retourne expression 2 si expression1 est vraie, ou expression 3 sinon.

Exemple : détermination de la valeur absolue

```
double dX = -8.4;
double dVal = (dX > 0) ? dX : -dX;
```

Opérateur sizeof()

`sizeof()` donne la taille en octets de l'opérande qui lui est associé. L'opérande peut être un type, une variable ou une valeur.

Exemples

`sizeof (double);` retourne 8.

Avec `short sV = 3; sizeof (s);` retourne 2.

`sizeof (4.5f)` retourne 4.

Opérateur ,

L'opérateur `,` permet de regrouper une liste d'expressions en une seule expression. Deux expressions séparées par une virgule sont évaluées de gauche à droite et l'ensemble a comme valeur celle de l'expression la plus à droite.

Exemple

`dC = (dA = 1, dB = 2);` affecte 1 à dA et 2 à dB, et affecte 2 à dC.

Ce type d'écriture, peu lisible, doit être évité.

Opérateur .

L'opérateur `.` est utilisé pour accéder aux membres (données ou fonctions) d'une classe, d'une structure ou d'une union. Exemple : `class X x; x.init (); x.done ();`

Opérateur ::

L'opérateur `::` est utilisé pour relier la définition d'une fonction à la déclaration d'une classe. Exemple :

```
class X {...}  int X::x()...
```

Autres opérateurs

Il existe d'autres opérateurs comme `new`, `delete`, `throw`, `->`, `..`, `[ ]`, `type&`, ... sur lesquels nous reviendrons plus loin.

Précédence des opérateurs

Dans une expression qui comporte plusieurs opérateurs, l'évaluation s'effectue suivant un ordre de priorité décroissant.

A chaque opérateur est associé une priorité appelée *précédence d'opérateur*.

Dans le tableau suivant, les opérateurs placés sur une même ligne sont d'égale *précédence*, les lignes étant disposées par *précédence décroissante*.

Opérateurs	Associativité
() [] { } -> .	gauche → droite
! ~ ++ -- - () * & sizeof()	droite → gauche
* / %	gauche → droite
+ -	gauche → droite
<< >>	gauche → droite
< <= > >=	gauche → droite
== !=	gauche → droite
&	gauche → droite
^	gauche → droite
	gauche → droite
&&	gauche → droite
	gauche → droite
? :	droite → gauche
= += -= *= /= %= >>= <<= &= = =	droite → gauche
,	gauche → droite

Tableau 12: précedence des opérateurs.